

3.20 Performans ve optimizasyon

Bu belge, Django kodunuzun daha verimli, daha hızlı ve daha az sistem kaynağı kullanarak çalışmasına yardımcı olabilecek teknikler ve araçlara genel bir bakış sunmaktadır.

3.20.1 Giriş

Genellikle ilk öncelik, beklenen çıktıyı üretmek için gerekli mantığı işleyen, çalışan bir kod yazmaktır. Ancak bazen bu, kodun istenildiği kadar verimli çalışması için yeterli olmayabilir. Bu durumda, kodun davranışını etkilemeden veya yalnızca minimum düzeyde etkileyerek performansını iyileştirecek bir şeye ve pratikte genellikle bir dizi şeye ihtiyaç duyulur.

3.20.2 Genel yaklaşımlar

Ne için optimize ediyorsunuz?

'Performans' derken ne demek istediğinizi net bir şekilde anlamanız önemlidir. Sadece bir metrik yoktur.

Hızın artırılması bir program için en bariz hedef olabilir, ancak bazen daha düşük bellek tüketimi veya veritabanı veya ağa daha az yük gibi performans iyileştirmeleri de aranabilir. Bir alandaki iyileştirmeler genellikle başka bir alanda daha iyi performans getirir, ancak her zaman değil; bazen biri diğerinin pahasına bile olabilir. Örneğin, bir programın hızındaki bir iyileşme daha fazla bellek kullanmasına neden olabilir. Daha da kötüsü, bu durum kendini kaybedebilir - hız iyileştirmesi o kadar hafıza açıyorsa ki sistem hafıza tükenmeye başlarsa, zarardan çok zarar vermiş olursunuz.

Aklınızda tutulması gereken başka takaslar da var. Kendi zamanınız değerli bir kaynaktır, CPU zamanından daha değerli. Bazı iyileştirmeler uygulamaya değer olmayacak kadar zor olabilir veya kodun taşınabilirliğini veya sürdürülebilirliğini etkileyebilir. Her performans iyileştirmesi çabaya değmez.

Yani, hangi performans iyileştirmelerini hedeflediğinizi bilmeniz gerekiyor ve ayrıca o yöne nişan almak için iyi bir sebebiniz olduğunu bilmelisiniz - ve bunun için şöyle şeyler gerekiyor:

Performans kıyaslaması

Sadece tahmin etmek ya da kodundaki verimsizliklerin nerede olduğunu varsaymak işe yaramaz

Django aletleri

`django-debug-toolbar`, kodunuzun ne yaptığı ve ne kadar zaman harcadığı hakkında içgörüler sağlayan çok kullanışlı bir araçtır. Özellikle sayfanızın oluşturduğu tüm SQL sorgularını ve her birinin ne kadar sürdüğünü gösterebilir.

Araç çubuğu için üçüncü taraf paneller de mevcuttur; örneğin önbellek performansı ve şablon render sürelerini raporlayabilir.

Üçüncü taraf hizmetleri

Sitenizin sayfalarının performansını uzaktan HTTP istemcisi perspektifinden analiz edip raporlayan, gerçek bir kullanıcının deneyimini simüle eden birçok ücretsiz hizmet vardır.

Bunlar kodunuzun iç kısımlarını raporlayamaz, ancak sitenizin genel performansına dair faydalı bir içgörü sağlayabilir; Django ortamında yeterince ölçülenemeyen yönler de dahil.

Benzer analiz yapan birkaç ücretli hizmet de vardır; bunlar arasında Django farkında olanlar da vardır ve kod tabanınızla entegre olup performansını çok daha kapsamlı profil eder.

Baştan doğru yapmak

Optimizasyon çalışmalarının bir kısmı performans eksikliklerini gidermeyi içerirken, bazı çalışmalar zaten yapacağınız işlere entegre edilebilir; bu, performansı iyileştirmeyi düşünmeye başlamadan önce bile benimsemeniz gereken iyi uygulamaların bir parçasıdır. Bu açıdan Python, çalışmak için mükemmel bir dildir, çünkü zarif görünen ve doğru hissettiren çözümler genellikle en iyi performansı gösterenlerdir. Çoğu beceride olduğu gibi, "doğru görünen" şeyin ne olduğunu öğrenmek pratik gerektirir, ancak en faydalı kılavuzlardan biri şudur:

Uygun seviyede çalış

Django, birçok farklı yaklaşım sunar, ancak bir şeyi belirli bir şekilde yapmanın mümkün olması, bunun en uygun yol olduğu anlamına gelmez.

Örneğin, bir koleksiyondaki öğe sayısını hesaplamak için aynı işlemi bir QuerySet'te, Python'da veya bir şablonda yapabileceğinizi görebilirsiniz. Ancak, bu işi daha yüksek seviyelerde yapmak neredeyse her zaman daha düşük seviyelerde yapmaktan daha hızlı olacaktır. Daha yüksek seviyelerde sistem, nesnelerle şu yollarla ilgilenmek zorundadır:

```
# Veritabanında QuerySet işlemi hızlı,  
# çünkü veritabanlarının en iyi yaptığı şey bu.  
my_bicycles.count()  
  
# Python nesnelerinin sayılması daha yavaş çünkü her halükarda bir  
# veritabanı sorgusu gerektiriyor ve  
# Python nesnelerinin işlenmesini de içeriyor.  
len(my_bicycles)
```

```
<!--  
Django şablon filtresi daha da yavaş çalışır,  
çünkü Python'da bunları sayması gerekecek ve şablon  
dilinin getirdiği ek yükler de buna neden olacaktır.  
-->  
{{ my_bicycles|length }}
```

Genel olarak, iş için en uygun seviye, kodlama konusunda rahat olunan en düşük seviyedir.

Not

Yukarıdaki örnek sadece açıklayıcı niteliktedir.

Öncelikle, gerçek hayatta, sayım işleminden önce ve sonra neler olduğunu göz önünde bulundurarak, o özel bağlamda en uygun yöntemin ne olduğunu belirlemeniz gerekir. Veritabanı optimizasyon belgesi, [şablonda saymanın daha iyi olacağı bir durumu](#) açıklamaktadır.

İkinci olarak, dikkate alınması gereken başka seçenekler de vardır: gerçek hayatta, QuerySet'in count() yöntemini doğrudan şablondan çağırarak `{{ my_bicycles.count }}` en uygun yöntem olabilir....

3.20.3 Önbellekleme

Bir değeri hesaplamak genellikle maliyetlidir (yani, kaynak tüketimi yüksek ve yavaştır), bu nedenle değeri bir sonraki ihtiyaç duyulduğunda kullanılmak üzere hızlı erişilebilir bir önbelleğe kaydetmenin büyük faydası olabilir.

Bu, Django'nun kapsamlı bir önbellekleme çerçevesi ve diğer küçük önbellekleme işlevlerini içerecek kadar önemli ve güçlü bir tekniktir.

Önbellek çerçevesi

Django'nun [önbellekleme çerçevesi](#), dinamik içeriği her istek için yeniden hesaplanmasına gerek kalmayacak şekilde kaydederek performans artışı için çok önemli fırsatlar sunar.

Kolaylık sağlamak için Django, farklı önbellekleme ayrıntı düzeyleri sunar: belirli görünümünün çıktısını, yalnızca üretilmesi zor olan parçaları veya hatta tüm bir siteyi önbelleğe alabilirsiniz.

Önbellekleme uygulamak, kötü yazılmış olduğu için kötü performans gösteren kodu iyileştirmenin bir alternatifi olarak görülmemelidir.

cached_property

Bir sınıf örneğinin metodunu birden fazla kez çağırmak yaygın bir durumdur. Eğer bu fonksiyon maliyetli ise, bunu yapmak israf olabilir.

[cached_property](#) dekoratörü, bir özelliğin döndürdüğü değeri kaydeder; bu örnek üzerinde fonksiyon bir sonraki çağrıldığında, yeniden hesaplamak yerine kaydedilen değeri döndürür. Bunun yalnızca ``self`` 'i tek argüman olarak alan metotlarda çalıştığını ve metodu bir özelliğe dönüştürdüğünü unutmayın.

Bazı Django bileşenlerinin de kendi fonksiyonel olarak önbellekleme dekoratörleri vardır; bunlar, ilgili bileşenlere ait bölümlerde aşağıda ele alınmaktadır.

3.20.4 Tembelliği anlamak

Tembellik, önbelleğe tamamlayıcı bir stratejidir. Önbellekleme, sonuçları kaydederek yeniden hesaplamayı önler; tembellik, hesaplamayı gerçekten gerekli olana kadar geciktirir.

Tembellik, bir şeye ortaya çıkmadan önce ya da hatta ortaya çıkmadan önce onlara atıfta bulunmamıza olanak tanır. Bunun birçok kullanımı vardır.

Örneğin, [tembel çeviri](#) hedef dil bilinmeden önce kullanılabilir, çünkü çevrilmiş dizi gerçekten gerekli olana kadar gerçekleşir, örneğin render edilmiş bir şablonda.

Tembellik aynı zamanda işten kaçınmaya çalışarak çaba harcamanın bir yoludur. Yani, tembelliğin bir yönü, yapılması gereken bir şey olmadan hiçbir şey yapmamaktır, çünkü bu aslında gerekli olmayabilir. Bu nedenle tembelliğin performans etkileri olabilir ve ilgili iş ne kadar pahalıysa, tembellikten o kadar çok kazanç elde edilir.

Python, özellikle üretici ve üretici [ifade](#) yapıları aracılığıyla tembel değerlendirme için birçok araç sağlar. Python'da tembellik üzerine okumak, kodunuzdaki tembel kalıplardan yararlanma fırsatlarını keşfetmek için faydalı olacaktır.

Django'daki tembellik

Django kendisi oldukça tembel. Bunun iyi bir örneği, bir QuerySet'in değerlendirilmesinde bulunabilir. [QuerySet'ler tembel](#). Böylece bir QuerySet oluşturulabilir, aktarılabilir ve diğer QuerySet örnekleriyle birleştirilebilir; ancak tanımladığı öğeleri almak için veritabanına yapılan bir gezinti yaşanmaz. Paylaşılan şey, veritabanından gerekecek öğelerin koleksiyonu değil, QuerySet nesnesidir.

Öte yandan, [bazı işlemler bir QuerySet'in değerlendirilmesini zorunlu kılacaktır](#). Bir QuerySet'in erken değerlendirilmesini önlemek, pahalı ve gereksiz bir veritabanı yolculuğundan tasarruf edebilir.

Django ayrıca [keep_lazy\(\)](#) dekoratör de sunmaktadır. Bu, tembel bir argümanla çağrılan bir fonksiyonun tembelce davranmasına izin verir ve sadece gerektiğinde değerlendirilir. Bu nedenle, tembel argüman - ki pahalı olabilir - kesinlikle zorunlu kılınana kadar değerlendirilmez.

3.20.5 Veritabanları

Veritabanı optimizasyonu

Django'nun veritabanı katmanı, geliştiricilerin veritabanlarından en iyi performansı elde etmelerine yardımcı olacak çeşitli yöntemler sunar. [Veritabanı optimizasyon dokümantasyonu](#), ilgili dokümantasyona bağlantıları bir araya getirir ve veritabanı kullanımınızı optimize etmeye çalışırken atılacak adımları özetleyen çeşitli ipuçları ekler.

Diğer veritabanı ile ilgili ipuçları

[Kalıcı bağlantıların](#) etkinleştirilmesi, veritabanı hesaplarına bağlantıları istek işleme süresinin önemli bir kısmını hızlandırabilir.

Örneğin, sınırlı ağ performansına sahip sanallaştırılmış hostlarda bu durum çok yardımcı oluyor.

3.20.6 HTTP Performansı

Ara Yazılım

Django, sitenizin performansını optimize etmeye yardımcı olabilecek birkaç faydalı [ara yazılım](#) parçası ile geliyor. Bunlar şunları içerir:

ConditionalGetMiddleware

Modern tarayıcıların ETag ve Son Değiştirilmiş başlıklara göre koşullu olarak yanıt almasını destekler. Ayrıca gerekirse bir ETag hesaplar ve ayarlar.

GZipMiddleware

Tüm modern tarayıcılar için yanıtları sıkıştırır, bant genişliği ve aktarım süresinden tasarruf sağlar. GZipMiddleware'in şu anda bir güvenlik riski olarak kabul edildiğini ve TLS/SSL tarafından sağlanan korumayı geçersiz kılan saldırılara karşı savunmasız olduğunu unutmayın. Daha fazla bilgi için [GZipMiddleware](#)'deki uyarıya bakınız.

Oturumlar

Önbelleğe alınmış oturumların kullanımı

[Önbellekli oturumların](#), veritabanı gibi daha yavaş bir depolama kaynağından oturum verilerini yükleme ihtiyacını ortadan kaldırarak performansı artırmanın bir yolu olabilir ve bunun yerine sık kullanılan oturum verilerini bellekte saklayabilir.

Static files

Tanım gereği dinamik olmayan statik dosyalar, optimizasyon kazanımları için mükemmel bir hedef oluşturur.

ManifestStaticFilesStorage

Web tarayıcılarının önbellege alma yeteneklerinden faydalanarak, ilk indirmeden sonra belirli bir dosya için ağ vuruşlarını tamamen ortadan kaldırabilirsiniz.

ManifestStaticFilesStorage, *statik dosyalar*ın dosya adlarına içerik bağımlı bir etiket ekler; böylece tarayıcılar uzun vadede önbellege kaydedebilir ve gelecekteki değişiklikleri kaçırmazlar - bir dosya değiştiğinde, etiket de değişir, böylece tarayıcılar varlığı otomatik olarak yeniden yükler.

“Minification”

Birçok üçüncü taraf Django aracı ve paketi, HTML, CSS ve JavaScript'i "minifika etme" imkanı sağlar. Gereksiz boşlukları, yeni satırları ve yorumları kaldırır, değişken isimleri kısaltır ve böylece sitenizin yayınladığı belgelerin boyutunu azaltır.

3.20.7 Şablon performansı

Şunu unutmayın:

- {% block %} kullanmak, {% include %} kullanmaktan daha hızlıdır
- çok sayıda küçük parçadan oluşturulmuş ağır parçalanmış şablonlar performansı etkileyebilir

Önbellekli şablon yükleyici

Önbellekli şablon yükleyicisini etkinleştirmek genellikle performansı büyük ölçüde artırır, çünkü her şablon her render edildiğinde derlemekten kaçınır

3.20.8 Mevcut yazılımların farklı sürümlerini kullanmak

Kullandığınız yazılımın farklı ve daha iyi performans gösteren sürümlerinin mevcut olup olmadığını kontrol etmek bazen faydalı olabilir.

Bu teknikler, zaten iyi optimize edilmiş bir Django sitesinin performans sınırlarını zorlamak isteyen daha gelişmiş kullanıcılara yöneliktir.

Ancak, performans sorunlarına sihirli çözümler değiller ve daha temel şeyleri doğru şekilde yapmayan sitelere marjinal kazançlardan daha iyi kazanç getirmeleri pek olası değil.

Not

Tekrar etmekte fayda var: zaten kullandığınız yazılıma alternatifler aramak, performans sorunlarının ilk cevabı değildir. Bu optimizasyon seviyesine ulaştığınızda, resmi bir kıyaslama çözümüne ihtiyacınız var.

Yeni genellikle - ama her zaman değil - daha iyidir

İyi bakımlı yazılımların yeni sürümünün daha az verimli olması oldukça nadirdir, ancak bakım sağlayıcılar her olası kullanım durumunu öngöremez - bu yüzden yeni sürümlerin muhtemelen daha iyi performans göstereceğini bilerseniz de, her zaman daha iyi performans göstereceğini varsaymayın.

Bu, Django'nun kendisi için de geçerli. Ardışık sürümler sistem genelinde birçok iyileştirme sundu, ancak uygulamanızın gerçek dünya performansını yine de kontrol etmelisiniz; çünkü bazı durumlarda değişiklikler daha iyi performans göstermekten çok daha kötü performans gösterebilir.

Python'un yeni sürümleri ve ayrıca Python paketleri de genellikle daha iyi performans gösterir - ama varsaymak yerine ölçmek gerekir.

Not

Belirli bir sürümde alışılmadık bir performans sorunu yaşamadığınız sürece, yeni sürümde genellikle daha iyi özellikler, güvenilirlik ve güvenlik görürsünüz ve bu avantajlar, kazanabileceğiniz veya kaybettiğiniz performanslardan çok daha önemlidir.

Django'nun şablon diline alternatifler

Neredeyse tüm durumlarda, Django'nun yerleşik şablon dili gayet yeterli. Ancak, Django projenizdeki darboğazlar şablon sisteminde yatıyor gibi görünüyorsa ve bunu düzeltmek için diğer fırsatları kullandıysanız, üçüncü taraf bir alternatif çözüm olabilir.

Jinja2, özellikle hız açısından performans iyileştirmeleri sunabilir.

Alternatif şablon sistemleri, Django'nun şablon dilini ne ölçüde paylaştıklarına göre değişiklik gösterir.

Not

Şablonlarda performans sorunları yaşıyorsanız, yapmanız gereken ilk şey tam olarak nedenini anlamaktır. Alternatif bir şablon sistemi kullanmak daha hızlı olabilir, ancak aynı kazanımlar da bu zahmete girmeden erişilebilir olabilir - örneğin, şablonlarınızdaki pahalı işlem ve mantık görünümünüzde daha verimli yapılabilir.

Alternatif yazılım uygulamaları

Kullandığınız Python yazılımının aynı kodu daha hızlı çalıştırabilen farklı bir uygulamada sunulup sunulmadığını kontrol etmek faydalı olabilir.

Ancak: iyi yazılmış Django sitelerindeki çoğu performans sorunu Python yürütme seviyesinde değil, verimsiz veritabanı sorgulama, önbellekleme ve şablonlarda olur. Kötü yazılmış Python koduna güveniyorsanız, performans sorunlarınızın daha hızlı çalıştırılmasıyla çözülmesi pek olası değildir.

Alternatif bir uygulama kullanmak uyumluluk, dağıtım, taşınabilirlik veya bakım sorunlarına yol açabilir. Standart olmayan bir uygulamayı benimsemeden önce, uygulamanızın potansiyel riskleri aşacak kadar yeterli performans kazanımı sağladığından emin olmanız gerekir.

Bu uyarıları göz önünde bulundurarak, şunları bilmelisiniz:

PyPy

PyPy, Python'un kendisinde bir uygulamasıdır ('standart' Python uygulaması C dilindedir). PyPy, genellikle ağır uygulamalar için önemli performans artışları sağlayabilir.

PyPy projesinin temel hedeflerinden biri, mevcut Python API'leri ve kütüphaneleriyle **uyumluluktur**. Django, desteklenen Python sürümlerine karşılık gelen PyPy sürümleriyle uyumludur, ancak güvendiğiniz diğer kütüphanelerin uyumluluğunu kontrol etmeniz gerekir

Python kütüphanelerinin C uygulamaları

Bazı Python kütüphaneleri de C dilinde uygulanır ve çok daha hızlı olabilir. Aynı API'leri sunmayı hedefliyorlar. Uyumluluk sorunları ve davranış farklılıkları bilinmediği (ve her zaman hemen fark edilmediği) not edin.

Sorular:

Django'da bir koleksiyondaki (örneğin bir QuerySet içindeki) öğelerin sayısını hesaplamak için en verimli yöntem aşağıdakilerden hangisidir ve bunun temel nedeni nedir?

- A) `len(my_bicycles)`; çünkü Python'un yerleşik fonksiyonları her zaman veritabanı işlemlerinden daha hızlı çalışır.
- B) `{{ my_bicycles|length }}`; çünkü şablon filtreleri sunucu tarafındaki yükü azaltarak tarayıcı performansını optimize eder.
- C) `my_bicycles.count()`; çünkü bu işlem veritabanı seviyesinde gerçekleştirilir ve veritabanları bu tür hesaplamalarda çok daha hızlıdır.
- D) `cached_property` kullanmak; çünkü bu dekoratör her durumda veritabanı sorgularını tamamen ortadan kaldırır. t

Cevap C'dir

Django'da "Tembellik" (Laziness) ne anlama gelir ve QuerySet'lerin tembel olması geliştiriciye ne kazandırır?

Tembellik: Bir işlemin (örneğin veritabanından veri çekmenin), sonuçlarına gerçekten ihtiyaç duyulana kadar ertelenmesidir. Kısacası; "lazım olana kadar hiçbir iş yapmamak" prensibidir.

QuerySet'lerin avantajları: QuerySet'ler tembel olduğu için onları oluşturduğunuzda veritabanına hemen gidilmez. Bu sayede geliştirici, sorguyu arka arkaya filtreleyebilir veya

değiştirebilir; veritabanı ile olan asıl iletişim sadece en son ve gerekli olduğunda yapıldığı için gereksiz işlem yükünden ve zaman kaybından tasarruf edilir.